

Unit 5

FIFOs and Streaming Interfaces

ECE-GY 6463: ADVANCED HARDWARE DESIGN

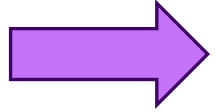
PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

Skill	You will be able to
Work out FIFO timing	Find the cycle each element is written, read and finished, and when a full FIFO stalls the producer
Size memory for a data stream	Compute a FIFO's element width and total size from its fields and its depth
Compare producer and consumer rates	Decide whether a consumer keeps up, when a stream finishes, and how burstiness sets the FIFO depth
Time a valid/ready handshake	Apply registered backpressure: when TREADY drops, and the cycle a stalled transfer completes
Describe AXI4-Stream	Find the cycles that carry a transfer, and where a packet framed by TLAST ends
Serialize and deserialize structured data	Pack a header into 32-bit words and write the code, moving fixed-point fields by their bit pattern

Each skill is assessed by problems in the LLM grader. The demos add what no problem tests yet: streaming kernels in Vitis HLS, and command-response messages with transaction IDs and error codes.

Outline



FIFOs

- AXI4-Stream: Pure Streaming
- Command-Response Pattern with FIFOs
- Implementing Command-Response IPs in Vitis

Recap: The Cost of a Register Interface

22

cycles of bus traffic per call

5

cycles of actual computation

1

call at a time: nothing is queued

PROBLEM 1: HANDSHAKE OVERHEAD

- Every value is its own AXI4-Lite transaction: address, data, response
- About **3 cycles** per register transfer
- Guaranteed delivery and flow control, paid for **per value**

PROBLEM 2: POLLING

- The processor starts the IP, then **asks repeatedly** whether it is done
- The IP sits **idle** while the processor writes inputs and reads outputs
- Neither side can work ahead of the other

This unit: put a **FIFO** between them, so each side runs at its own pace.

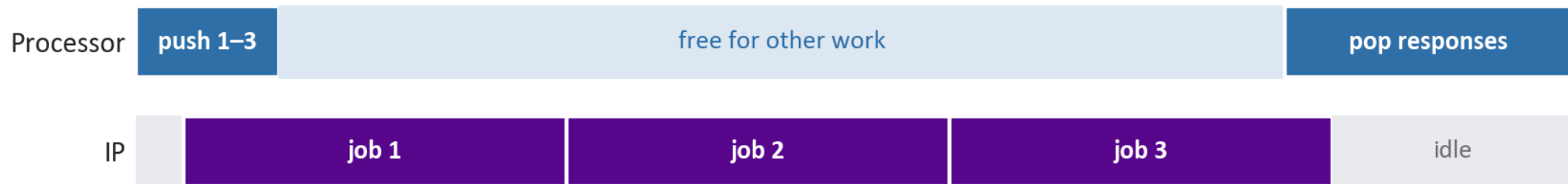
Figures measured in unit 4's scalar-function demo.

Where the Time Goes

Register interface



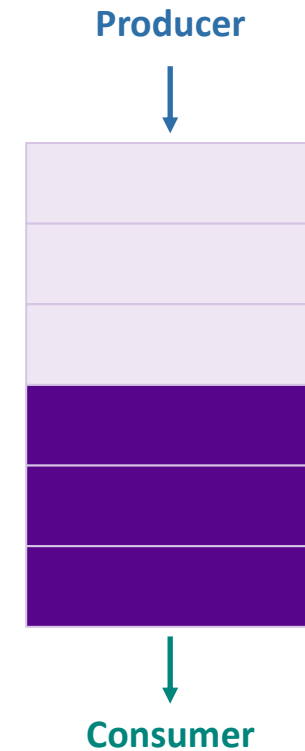
FIFO interface



A FIFO **decouples** the two sides in time: the processor queues work and leaves, and the IP runs job after job.

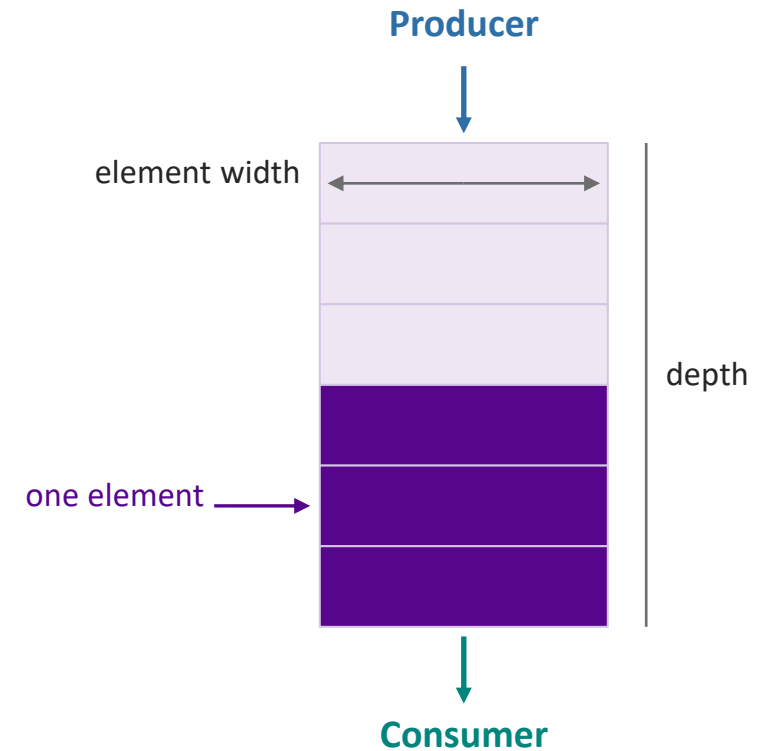
FIFO

- **FIFO** (first-in, first-out): a hardware queue
 - Stores elements from a **producer** in order of arrival
 - Returns them to a **consumer** in the same order
- **Decouples** producer and consumer timing
 - Each side runs at its own pace
- Built-in **flow control**
 - The producer waits while the FIFO is full
 - The consumer waits while it is empty
- Absorbs **bursts** of data, and enables **streaming**



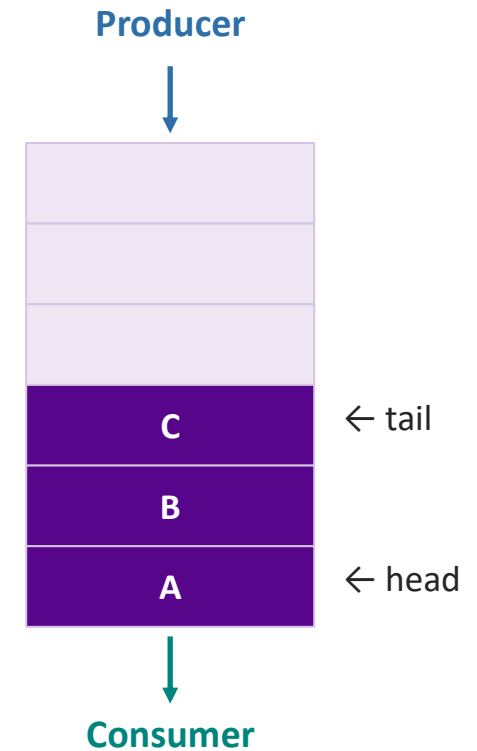
FIFO Terminology

- **Producer**: the module that writes to the FIFO
 - Each write adds an element
- **Consumer**: the module that reads from it
 - Each read removes an element
- **Element**: one item stored in the FIFO
- **Element width**: the size of each element, in bits or bytes
- **Depth**: the maximum number of elements
- **Size** = depth × element width



FIFO Order

- The producer writes A, then B, then C
- A is at the **head**: the oldest element
- C is at the **tail**: the newest
- The consumer reads from the head
 - It reads A, B, C: the order they were written
- No reordering, and no addresses
 - Contrast the random-access memory of unit 4



Sizing FIFOs

PRACTICE

A FIFO buffer stores commands with three fields. Here `ap_int<n>` is an integer with n bits. Assume the fields are packed with no padding.

- (a) What is the element width in bytes?
- (b) If the FIFO has a depth of 32 elements, what is the total size of the FIFO in bytes?

```
struct Command {  
    ap_int<16> command_id;  
    ap_int<32> operand1;  
    ap_int<32> operand2;  
};
```

Also in the LLM grader: Unit 5 › Sizing FIFOs

Sizing FIFOs

SOLUTION

(a) $16 + 32 + 32 = 80$ bits = $2 + 4 + 4$ bytes = **10 bytes**

(b) $32 \text{ elements} \times 10 \text{ bytes} = \mathbf{320 \text{ bytes}}$

Size = depth $\times$ element width

One element, no padding

id 2 B	operand1 4 B	operand2 4 B
-----------	-----------------	-----------------

Common errors

Leaving the width in bits: 80.

Padding command_id to 4 bytes: 12. The question rules that out.

Also in the LLM grader: Unit 5 › Sizing FIFOs

FIFO Timing

PRACTICE

A producer writes A, B and C into a FIFO at cycles 0, 1 and 6. An element can be read on the cycle after it is written.

After reading an element in cycle R, the consumer processes it in R+1, and can read the next one in R+2 at the earliest. It starts idle and reads as soon as it can.

At which cycle does the consumer read:

- (a) A?
- (b) B?
- (c) C?

Work it cycle by cycle

Cycle	Producer	FIFO after	Consumer
0	write A		
1	write B		
2	—		
3	—		
4	—		
5	—		
6	write C		
7	—		

Also in the LLM grader: Unit 5 › FIFO timing

FIFO Timing

SOLUTION

- (a) A is read at **cycle 1**, the first cycle after it is written
- (b) B is in the FIFO from cycle 2, but the consumer is processing A: read at **cycle 3**
- (c) The consumer is free from cycle 5, but C arrives at 6: read at **cycle 7**

Two limits

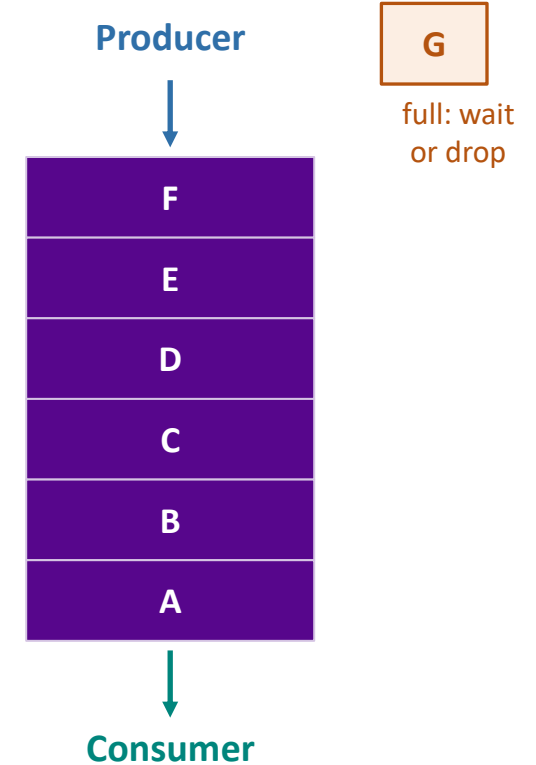
An element waits for a busy consumer (B), or the consumer waits for an element (C).

Cycle	Producer	FIFO after	Consumer
0	write A	[A]	idle
1	write B	[B]	read A
2	—	[B]	process A
3	—	[]	read B
4	—	[]	process B
5	—	[]	idle
6	write C	[C]	idle
7	—	[]	read C

Also in the LLM grader: Unit 5 › FIFO timing

FIFO Overflow

- The producer has an element, but the FIFO is **full**
- **Blocking write**
 - The FIFO deasserts READY; the producer waits and retries
 - The default for hardware interfaces, AXI4-Stream included
 - Also called a stall, backpressure or flow control
- **Drop**
 - An element is discarded: the new one (tail drop), or one already queued
 - Common in network routers



FIFO Write Stall

PRACTICE

A producer has elements A, B, C and D, available for writing at the earliest in cycles 0, 1, 2 and 3.

- The FIFO holds **2 elements**. A write can occur only if the FIFO was not full after the previous cycle; otherwise the producer **stalls** and retries each cycle.
- An element written in cycle N can be read, and removed, in N+1 at the earliest.
- A read in cycle R is processed in R+1 and R+2; the next read is at R+3 at the earliest. The consumer starts idle.

For each element, give the cycle its write **succeeds**, and the last cycle of its processing:

(a) A (b) B (c) C (d) D

Work it cycle by cycle

For each cycle: what the producer does, what the FIFO holds at the end of it, and what the consumer does.

The FIFO state at the end of a cycle decides the next cycle's write.

Also in the LLM grader: Unit 5 › FIFO write stall

FIFO Write Stall

SOLUTION

Cycle	Producer	FIFO after	Consumer
0	write A	[A]	idle
1	write B	[B]	read A
2	write C	[B, C] full	process A
3	D stalls	[B, C] full	process A
4	D stalls	[C]	read B
5	write D	[C, D] full	process B
6	—	[C, D] full	process B
7	—	[D]	read C
8–9	—	[D]	process C
10	—	[]	read D
11–12	—	[]	process D

Write succeeds · processing ends

- (a) A: **cycle 0 · cycle 3**
- (b) B: **cycle 1 · cycle 6**
- (c) C: **cycle 2 · cycle 9**
- (d) D: **cycle 5 · cycle 12**

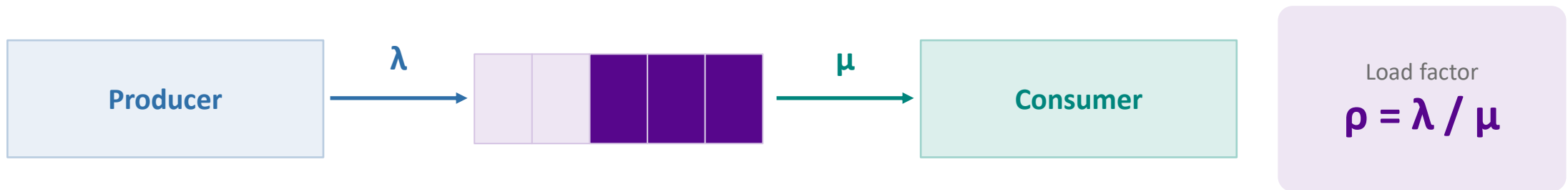
Why not cycle 4 for D?

B is read in cycle 4, but the write decision uses the FIFO state after cycle 3, which is full. The slot freed in cycle 4 is usable in cycle 5.

Also in the LLM grader: Unit 5 › FIFO write stall

Producer and Consumer Rates

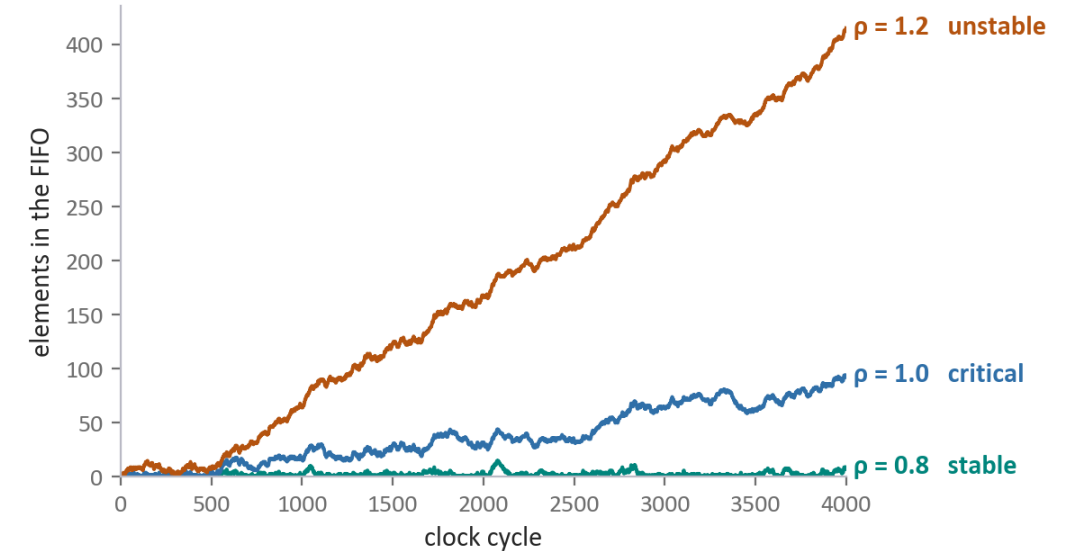
- **Producer rate** λ : the average number of elements written per unit time
 - Per clock cycle, per second, ... If the producer sends jobs, it is the workload
- **Consumer rate** μ : the maximum number of reads per unit time
 - If each element must be processed, it is the consumer's throughput
 - With a varying cost per element: 1 / the **average** time per element



ρ compares the averages. Whether the FIFO stays small depends on ρ , and also on how bursty the arrivals are: next.

Load Factor and Stability

Load factor	In hardware	What the FIFO does
$\rho < 1$ stable	Under-utilized: headroom	Drains between bursts. Short-term stalls or drops are still possible
$\rho = 1$ critical	Fully utilized	No headroom. Only perfectly regular traffic keeps up; with any randomness the backlog keeps growing
$\rho > 1$ unstable	Over-subscribed	Grows without bound. With backpressure the producer falls ever further behind; without it, elements are dropped



Random arrivals and departures, same random seed, unbounded FIFO. Queueing theory names the three regimes.

Throughput and FIFO Depth

PRACTICE

A vision accelerator processes frames one at a time. Each frame takes **5 ms**; on the **10%** of frames with an object of interest it takes an extra 100 ms, **105 ms** in all.

- (a) What is the maximum frame rate the accelerator can sustain without its input FIFO growing without bound?
- (b) Frames now arrive at **50 frames/s**, one every 20 ms. Rank these patterns of expensive frames by how large the input FIFO typically needs to be, smallest first, and explain:
 - (i) Uniform: every 10th frame — 0, 10, 20, ...
 - (ii) Bursty: runs of 10 — frames 0–9, then 100–109, then 200–209, ...
 - (iii) Random: each frame independently, with probability 10%

Also in the LLM grader: Unit 5 › Accelerator throughput and FIFO depth

Throughput and FIFO Depth

SOLUTION

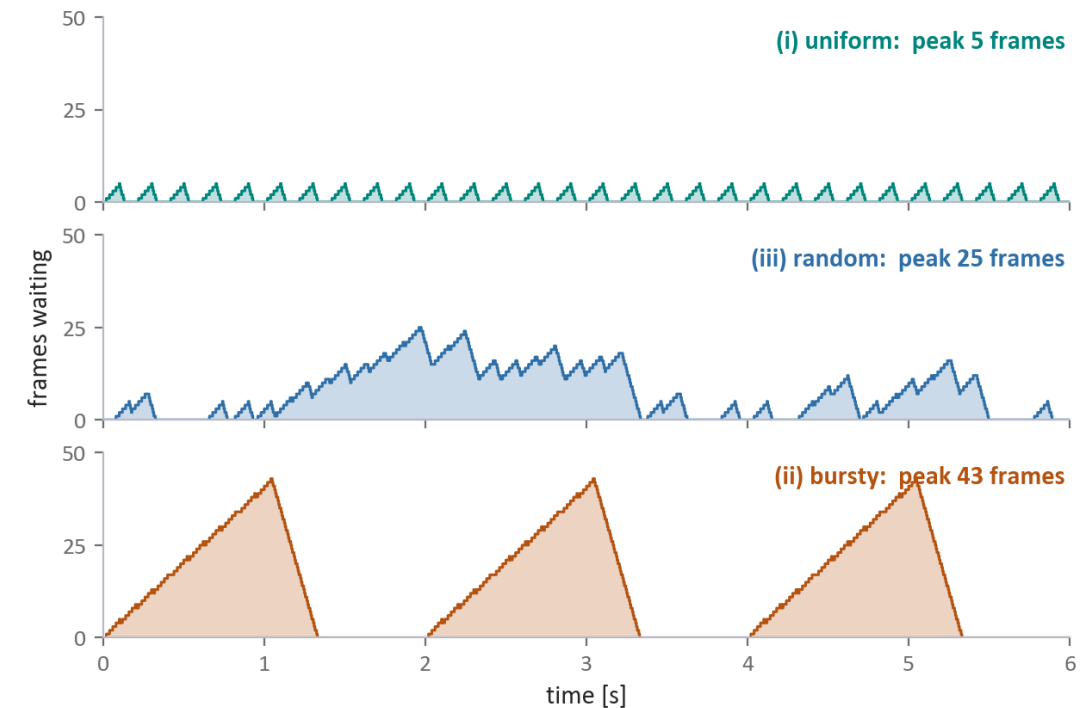
(a) Average time: $0.9 \times 5 + 0.1 \times 105 = 15$ ms
Maximum rate: $1 / 15$ ms $\approx$ **66.7 frames/s**

(b) (i) uniform < (iii) random < (ii) bursty

The average load is the same in all three. What differs is how the expensive frames cluster, and a cluster builds a backlog the FIFO must hold while the accelerator falls behind.

Uniform: about $105 / 20 \approx 5$ frames arrive during one expensive frame, and drain before the next.

Bursty: $10 \times 105 = 1050$ ms, while about 52 frames arrive: a backlog of 40–50.



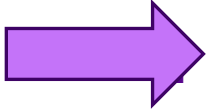
Simulated at 50 frames/s. The random pattern is one draw: its peak varies from run to run, and is unbounded in principle.

Also in the LLM grader: Unit 5 › Accelerator throughput and FIFO depth

Queueing Theory

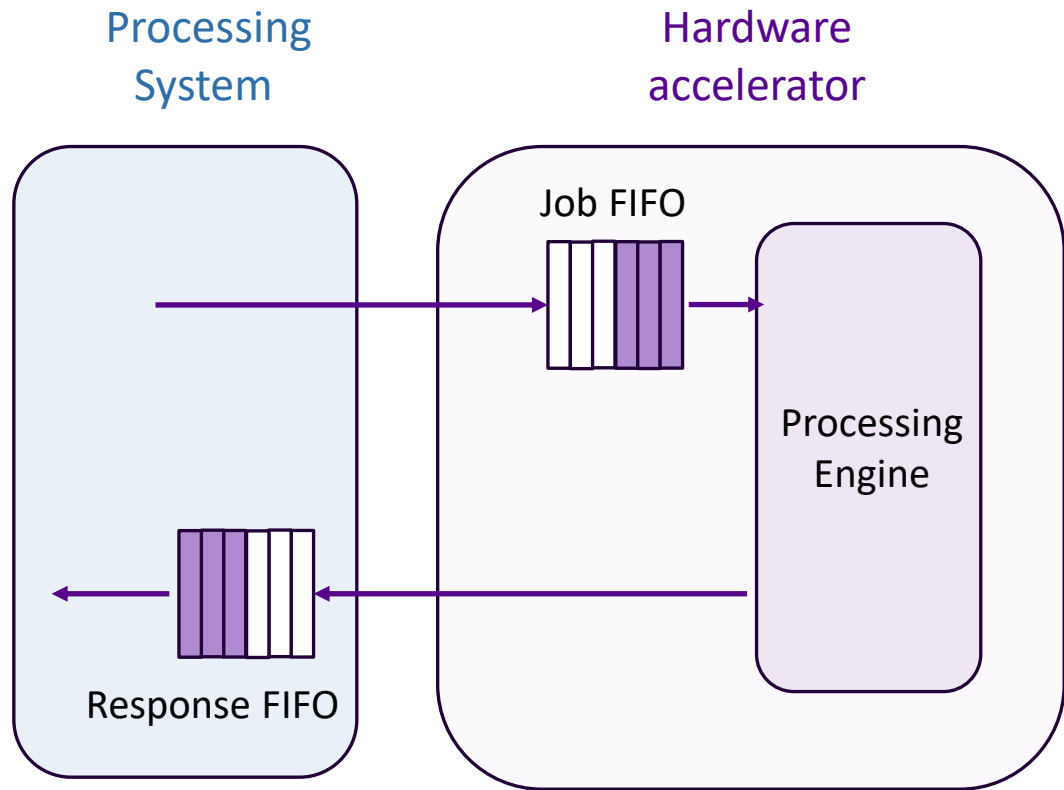
- Models arrivals and departures probabilistically
- Used widely in networking research
 - Determine probabilities of buffer overflow
 - Optimal routing and drop policies
 - Also used in network on chip
- But requires mathematics
 - Markov chains, probability
- Take NYU's networking theory class if interested

Outline

- FIFOs
-  AXI4-Stream: Pure Streaming
- Command-Response Pattern with FIFOs
- Implementing Command-Response IPs in Vitis

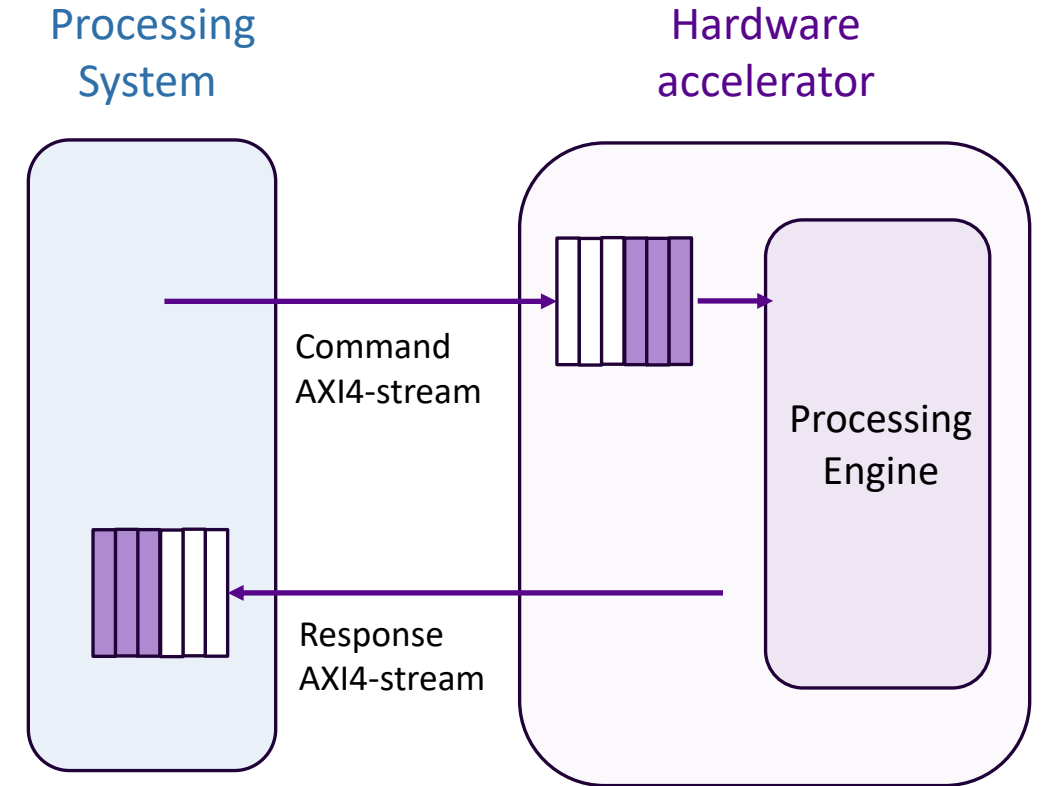
Solution: Streaming FIFO Interface

- Consider a PS sending jobs to accelerator
- Use FIFOs for jobs and responses
- Decouples for timing
 - Transfer of data of the PS
 - Processing of data in the HW
- Enables efficient burst transfers
 - Many elements can be transferred at once
 - Reduces handshaking overhead
- Both PS and HW run at their own pace
- If some jobs need a lot of time
 - Job FIFO buffers the jobs to process



AXI4-Stream

- Industry standard streaming protocol
 - Available as the interface to many Xilinx IP blocks
- Pre-built in Vitis HLS
 - All FIFO management is done for you
- Part of the AXI4 family:
 - AXI4-Lite: Covered last unit
 - AXI4-Stream: This unit
 - AXI4 (full, memory-mapped): later
- Core signals: TDATA, TVALID and TREADY
- Optional signals:
 - TLAST, marking the end of a packet: later in this unit
 - TKEEP, TSTRB, TID, TDEST, TUSER: not covered here



AXI4-Stream Terminology

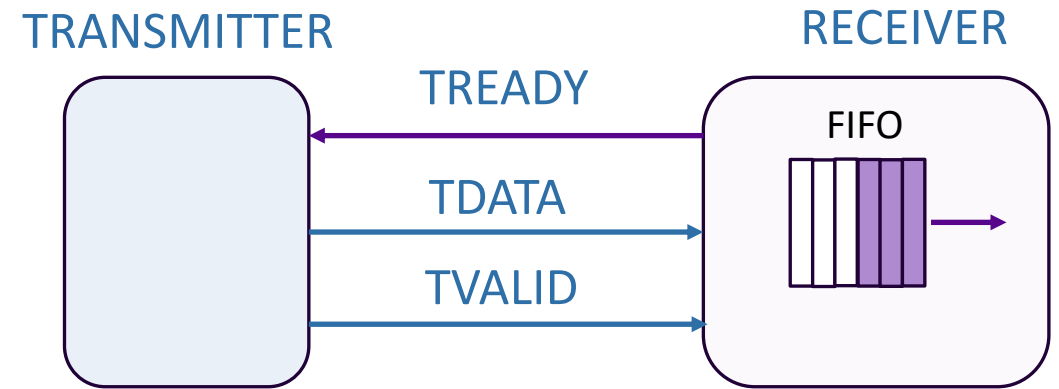


- Each AXI4-Stream is uni-directional
- Transmitter is the producer / sender
 - Could be PS sending jobs or HW accelerator returning responses
- Receiver is the consumer
 - Could be PS receiving response or HW accelerator receiving jobs
- FIFO is typically realized on the receiver side

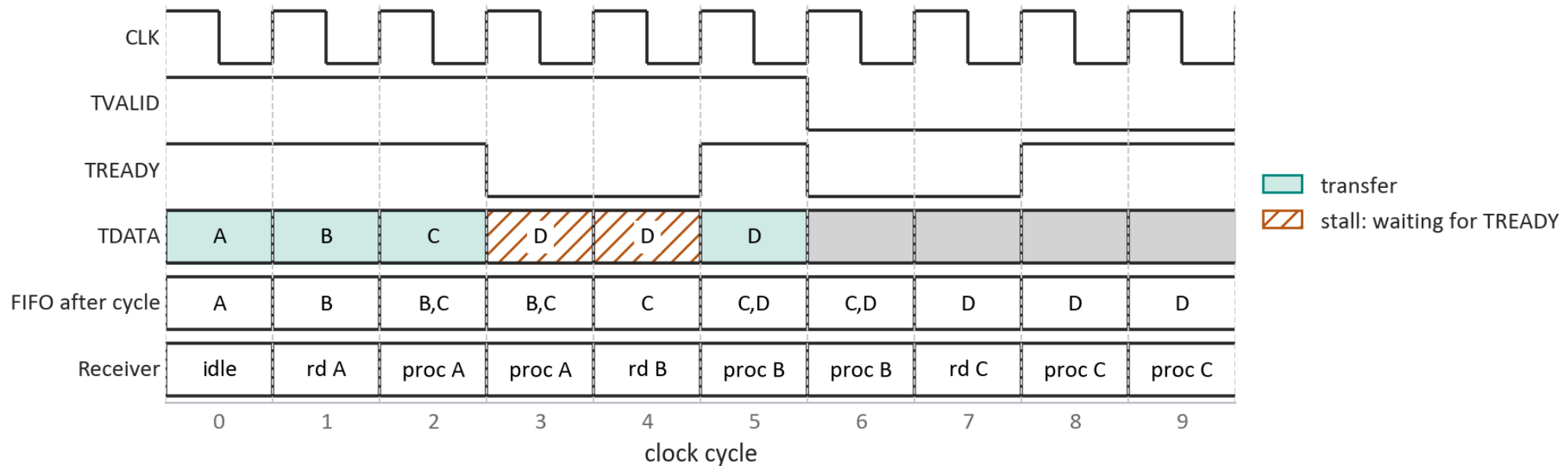
Called master and slave in older texts, and still in Vitis.

AXI4-Stream Protocol

- Receiver asserts TREADY when it can accept data
 - Typically, when FIFO has space
- On any cycle it has data, the transmitter:
 - Places data on TDATA
 - Asserts TVALID
- If TREADY=TVALID=1, there is a transfer
 - Otherwise it holds TVALID and TDATA until TREADY=1
- Receiver processes FIFO at its own pace
 - Backpressure to transmitter via TREADY
- At most one element transfers per cycle



Example: Backpressure on AXI4-Stream



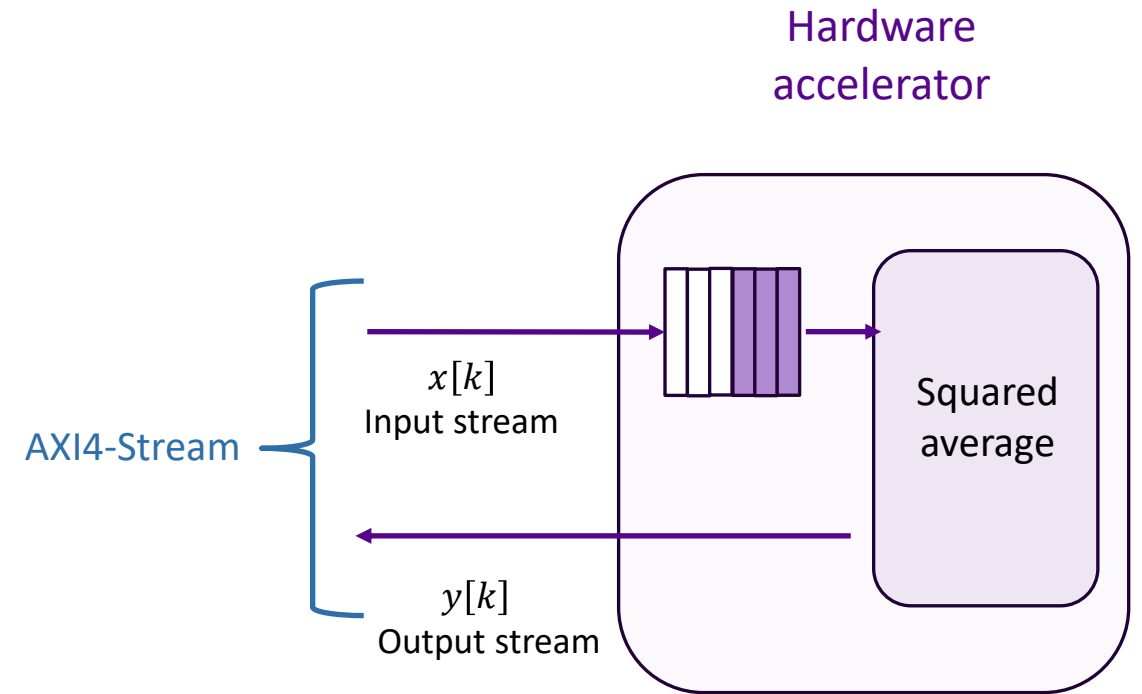
- The FIFO write-stall problem, on the wires: A–D offered at cycles 0–3, FIFO depth 2
- **Registered TREADY**: high in a cycle if the FIFO was not full after the previous one
- D waits with **TVALID held high** until TREADY returns in cycle 5

TREADY high with TVALID low (cycles 8–9) is not a transfer: the receiver is ready, the transmitter has nothing to send.

Homework in the LLM grader: Unit 5 › AXI4-Stream timing, AXI4-Stream timing with stalls, Non-pipelined slave throughput

Example: Streaming Squared Average

- Given:
 - Input stream $x[k]$
- Compute output stream:
 - $y[k] = \frac{1}{L}(x[k]^2 + \dots + x[k - L + 1]^2)$
 - L is a fixed window size
- Models an average power
- Use AXI4-Stream for $x[k]$ and $y[k]$



The Demo: a Streaming Filter

sdrangan.github.io/hwdesign — Demos › AXI4-Streaming

RUN THE WHOLE FLOW

```
cd demos/stream/avgfilt
python avgfilt_build.py --list-steps
python avgfilt_build.py --through verify_csim
python avgfilt_build.py
```

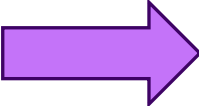
WHAT TO WATCH FOR

- The kernel has **no start or done**: two **axis** ports and **ap_ctrl_none**
- PIPELINE II=1: one sample in and one out **every cycle**
- The C simulation still calls the kernel **like a function**, and exits on an empty stream
- The timing diagram shows the handshake, the latency and the throughput

Measured in co-simulation: 200 samples out in 200 cycles, after a latency of 19 cycles, with no stalls.

This kernel runs free: no start, no done. Next: how to give it control.

Outline

- FIFOs
- AXI4-Stream: Pure Streaming
-  Command-Response Pattern with FIFOs
 - Implementing Command-Response IPs in Vitis

Limitations of Pure Streaming

Problem 1: Operations work on blocks

What the stream carries: samples



What the operation needs: blocks

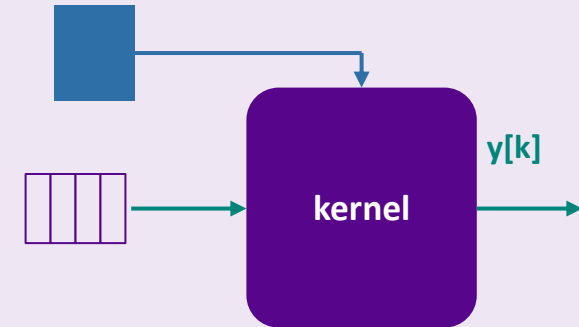


Pure streaming has **no block boundary**: where does one image end?

Problem 2: Control and data are hard to synchronize

control θ

data $x[k]$

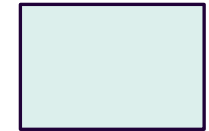
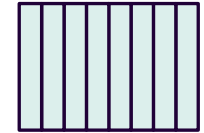


Control and data come by **different paths**: which sample does a new θ apply to?

Problem 1: Block-Based Operations

- Many problems naturally work on blocks
- Need to know boundaries
- Ex: Image processing
 - Each input is an image
 - Images have a first and last element
 - Scanned in some particular order (e.g. row-wise)
- Ex 2: Matrix multiply
 - Module needs two matrices
 - Each has to be read in some order
- Pure streaming is not natively good for these operations

Individual elements



Blocks of elements

Problem 2: Control-Data Synchronization

- This problem is more subtle
- Consider a parametrized streaming function:

$$y[k] = f(x[k], \theta)$$

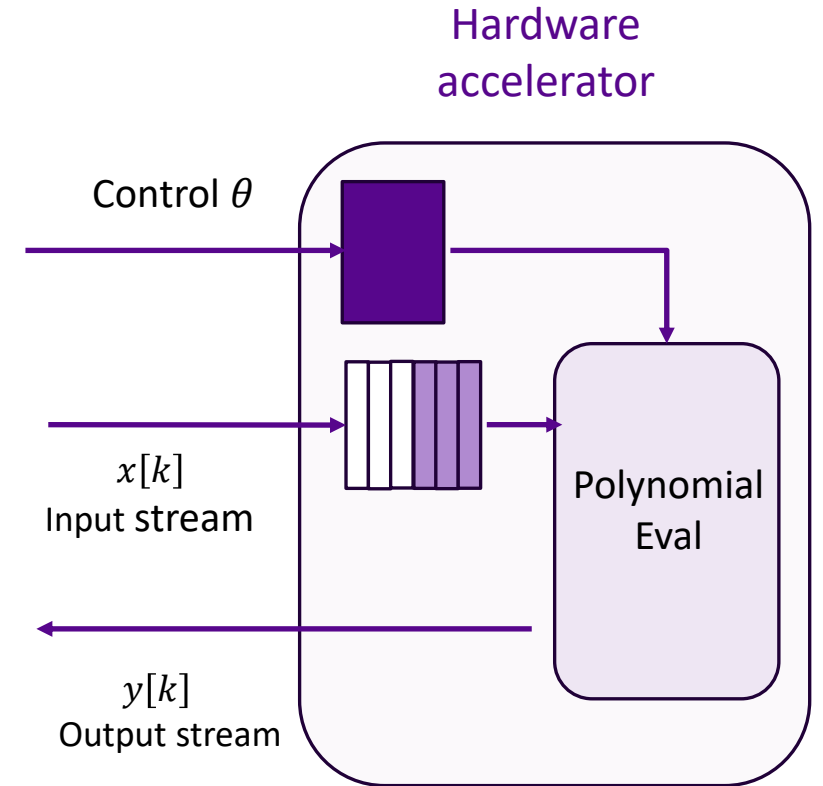
– θ : some parameter that changes slowly

- For example, streaming polynomial:

– $y[k] = a_0 + a_1x[k] + a_2x[k]^2$

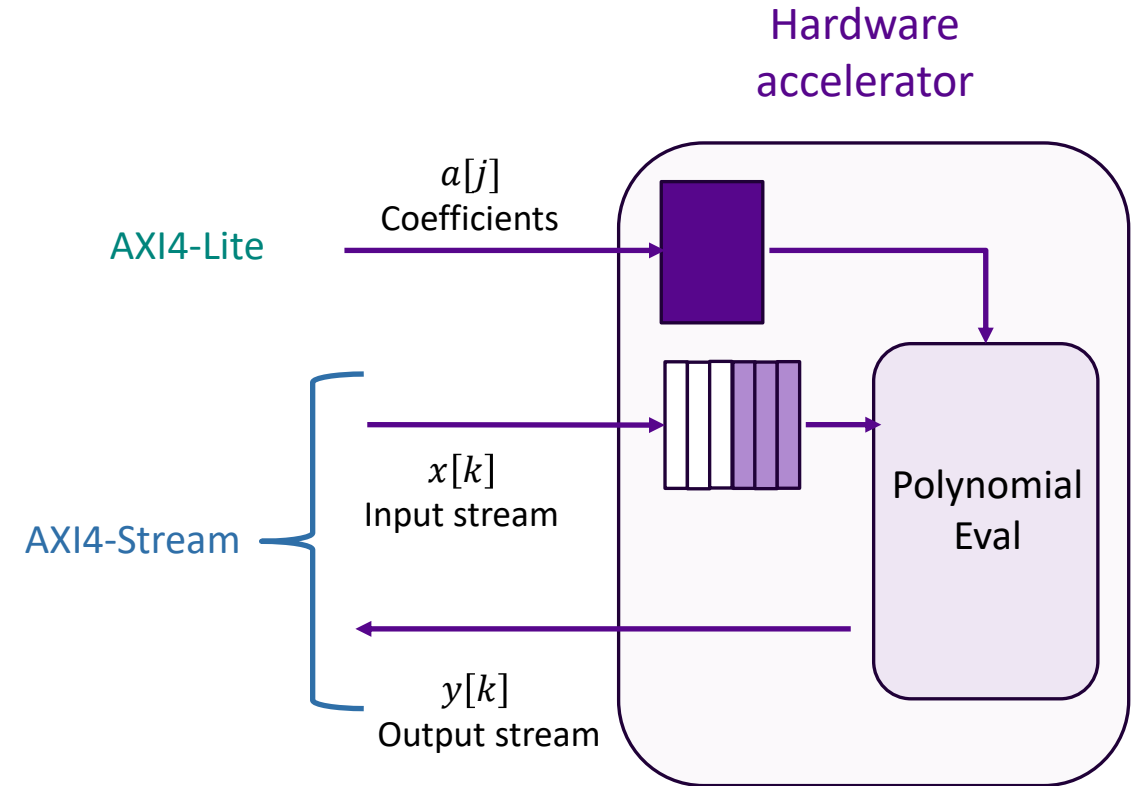
– $\theta = (a_0, a_1, a_2)$

θ is **control**: it changes rarely. $x[k]$ and $y[k]$ are **data**: a new value every cycle.



Solution: Mix AXI4-Lite and AXI4-Stream?

- Given:
 - Input stream $x[k]$
 - Polynomial coefficients $a[j]$ (change slowly)
- Compute output stream:
 - $y[k] = a[0] + a[1]x + \dots + a[d]x^d$
- Set coefficients via **AXI4-Lite**
 - Assume they change slowly
 - Do not need fast read / write
 - But benefit from continuous access
- Use **AXI4-Stream** for $x[k]$ and $y[k]$



A Potential HLS Module Definition

```
void stream_fun(  
    hls::stream<float> &in_stream,  
    hls::stream<float> &out_stream,  
    float a0,  
    float a1,  
    float a2  
) {  
    // AXI4-Stream interfaces  
    #pragma HLS INTERFACE axis port=in_stream  
    #pragma HLS INTERFACE axis port=out_stream  
  
    // Set AXI-Stream FIFO depths  
    #pragma HLS STREAM variable=in_stream depth=4  
    #pragma HLS STREAM variable=out_stream depth=4  
  
    // AXI4-Lite control interface for parameters and start signal  
    #pragma HLS INTERFACE s_axilite port=a0      bundle=CTRL  
    #pragma HLS INTERFACE s_axilite port=a1      bundle=CTRL  
    #pragma HLS INTERFACE s_axilite port=a2      bundle=CTRL  
    #pragma HLS INTERFACE s_axilite port=return bundle=CTRL  
}
```

Input and output streams

Coefficients

Declares that variables are AXI4-Stream
Does not state if transmitter or receiver.
Direction will be inferred

AXI4-Stream FIFO depth

Declares that coefficients are AXI4-Lite

Recall: Vitis Kernel Types

Vitis kernels come in two shapes. The difference is who decides when the kernel runs.

Host-Activated

```
#pragma HLS INTERFACE s_axilite port=return  
ap_ctrl_hs — the default
```

- Explicitly started by the processor
- Signals completion with a status bit, and can raise an interrupt
- The closest thing to a **function call** in hardware
- But the processor writes every input and reads every output
- Not how most production designs move data

Unit 4's model: simple, but control can change only between jobs.

Free-Running Task

```
#pragma HLS INTERFACE ap_ctrl_none port=return  
ap_ctrl_none
```

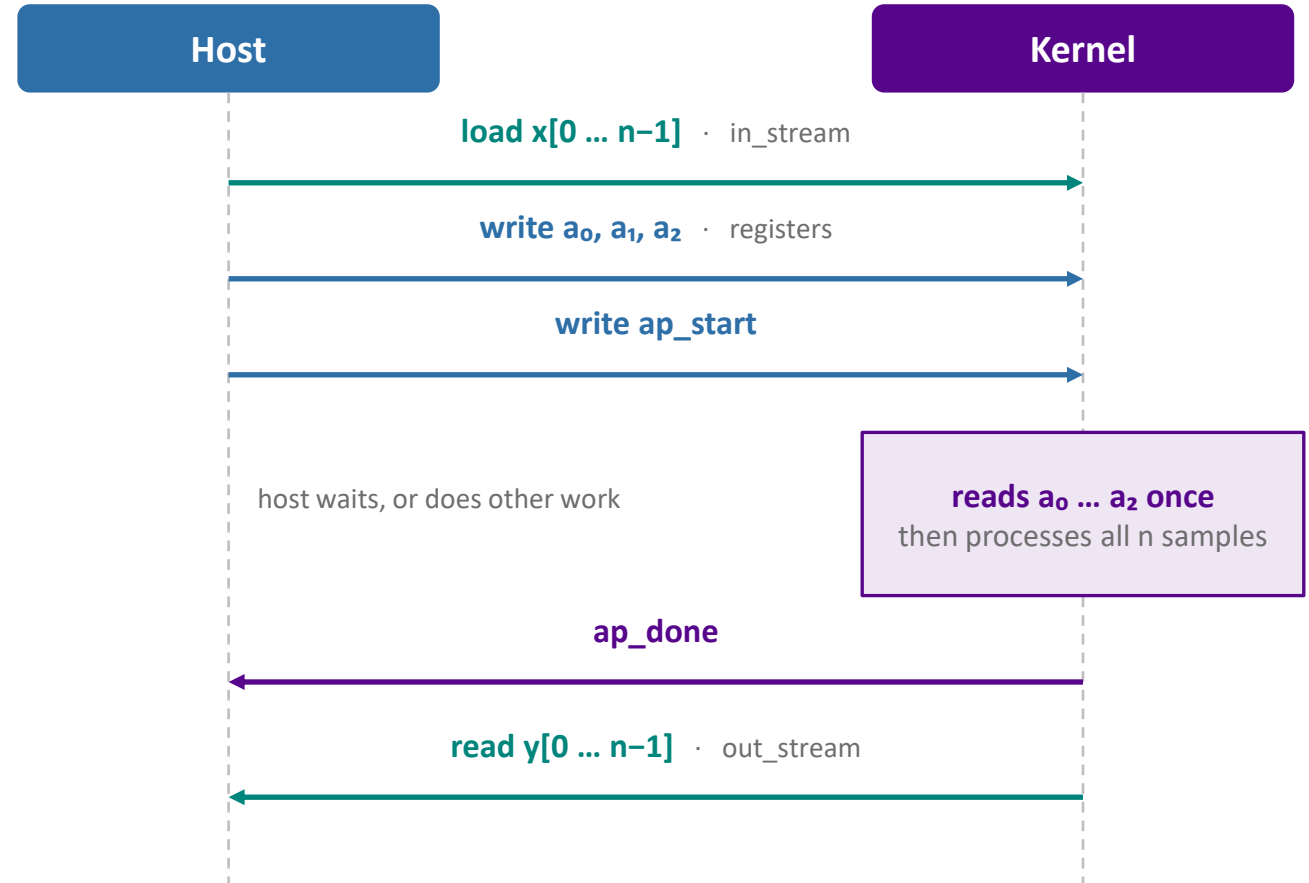
- Runs continuously, waiting on data
- Operates without the processor once started
- Enables far greater **concurrency**
- Data arrives on a stream, not in registers
- How real accelerators are built

The streaming filter is one. Next: how to give it control.

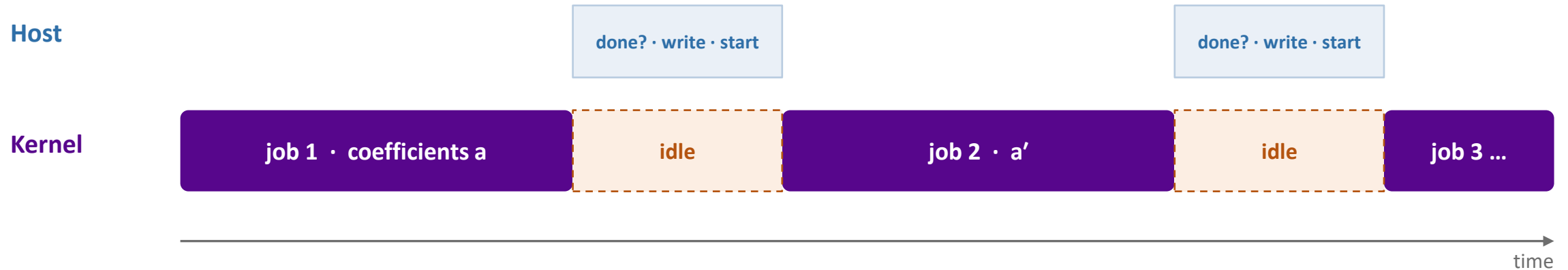
Operation Requires Host Activation

- The host runs the kernel **one job at a time**
- Control is read **once**, at the start, and applies to every sample in the job
- Safe: every register write happens **before** **ap_start**

The same protocol as unit 4's scalar function, with streams for the data.

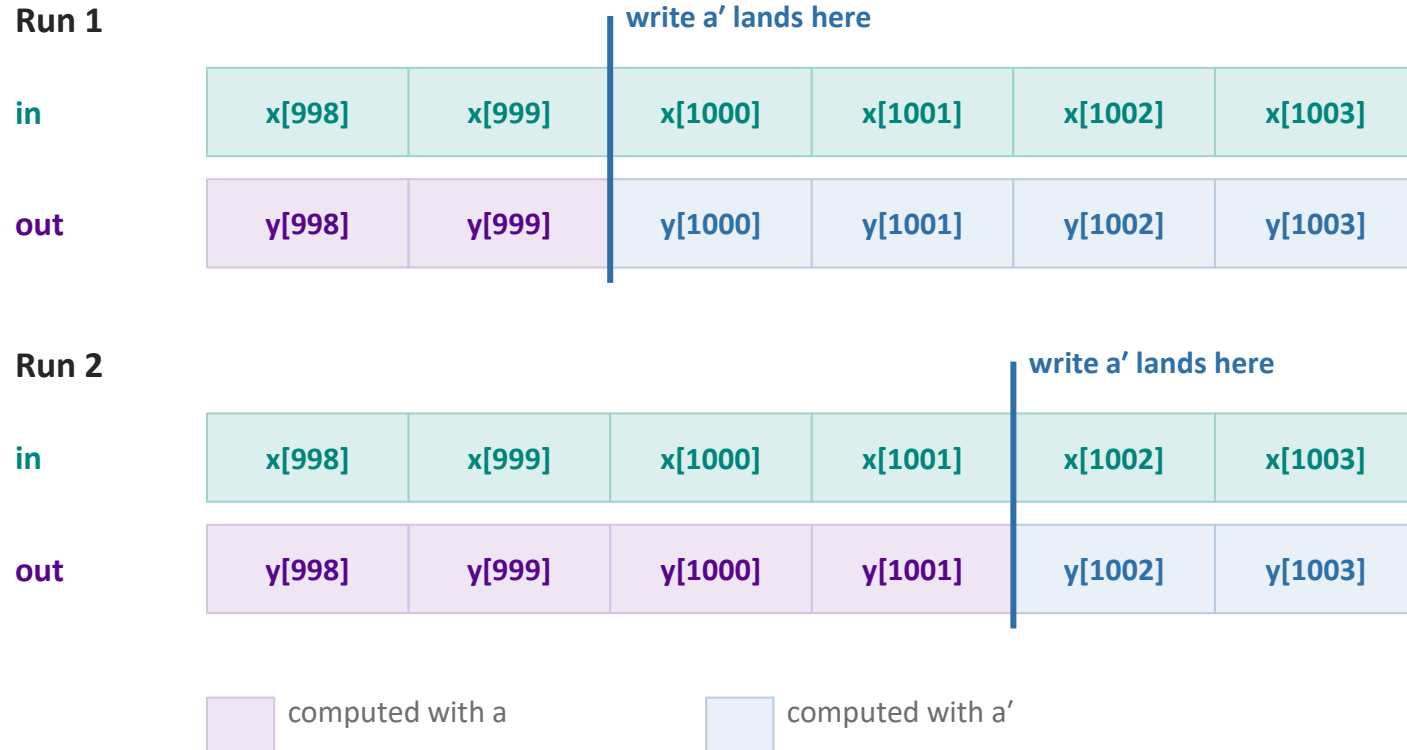


Problem with Host Activation



- Control can change only **between** jobs, never within one
- Every change costs a stop, a register write and a restart: the kernel sits **idle**, and the handshake and polling overhead from the start of this unit comes back

Why Not Free-Running?



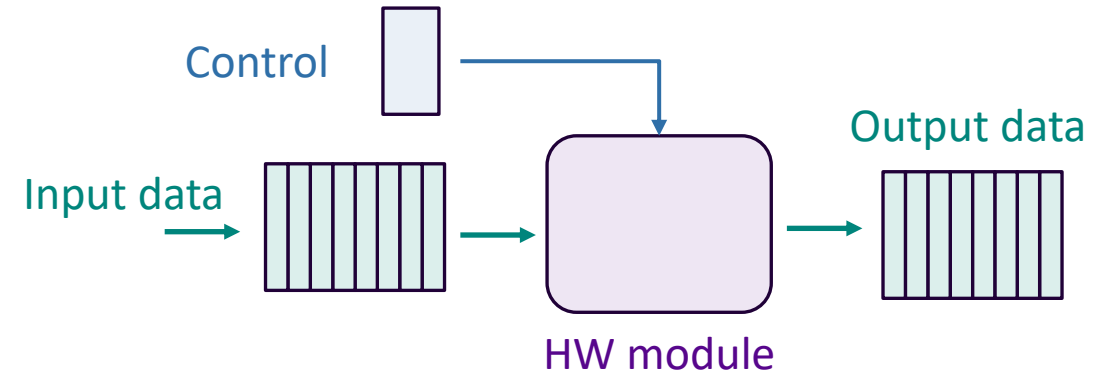
Same inputs, same write: different outputs.

- A register write to a running kernel lands “around now”
- Which sample sees it depends on **timing**: bus traffic, stalls, how full the FIFO is
- “Apply at sample $k = 1000$ ” needs extra logic to line the two paths up
- Instead: put the control **in the stream**

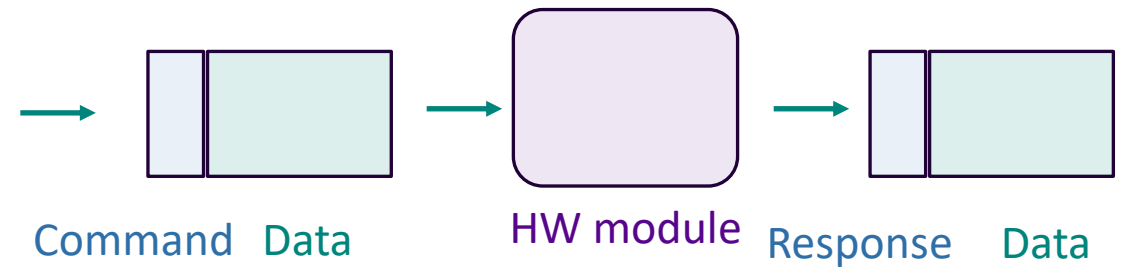
The FIFO Command-Response Pattern

- A **command** and its data arrive on one stream
- The **response** and the output data leave on another
- A command applies to the samples that follow it: no “apply at sample k”
- Nothing is written from the side, so the kernel can **run free**

BEFORE: CONTROL ON A SIDE PATH



AFTER: CONTROL IN THE STREAM



Why FIFOs Only?

cycle 0 1 2 3 4 5 6 7 8 9

Run A: no stalls

in	cmd	x0	x1	x2						
out				y0	y1	y2				

Run B: the input stalls

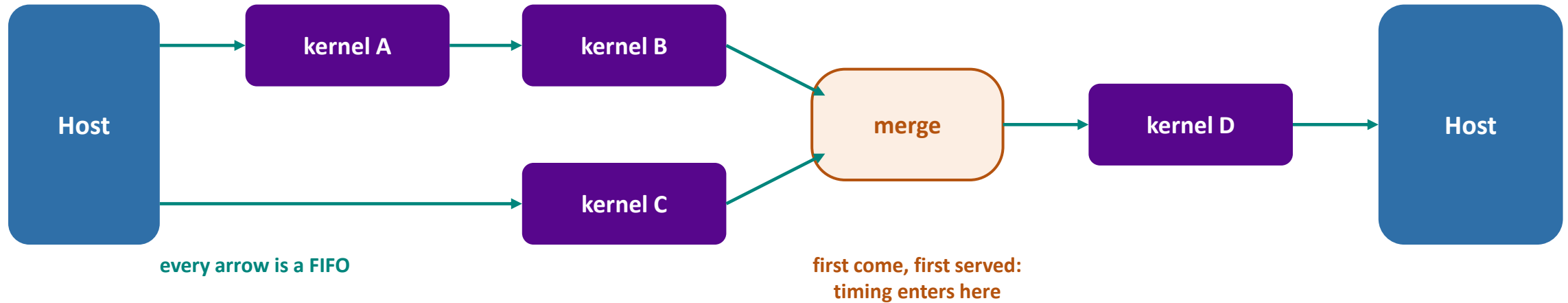
in	cmd		x0			x1	x2			
out					y0			y1	y2	

Both runs output y0, y1, y2 computed with the same command

Different cycles, same results.

- Every input, control included, is a FIFO read **in order**
- Outputs depend only on the **order and values** of the inputs
- Stalls, FIFO depth and latency change **when** an output appears, not **what** it is
- Timing comes back only where you put it: next slide

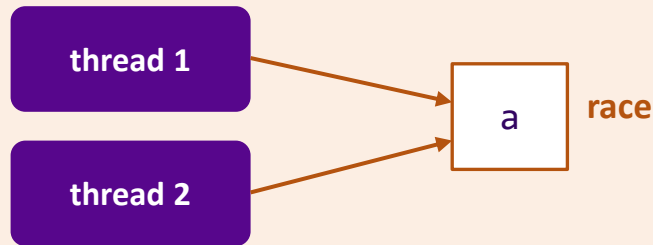
Multiple Kernels



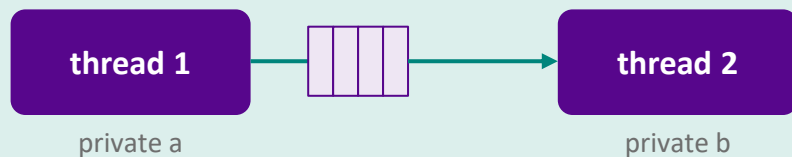
- This unit uses one kernel. For one kernel, registers written before a start would also work
- Several free-running kernels have no single start to hide behind: **streams only**
- A first-come merge or a shared memory port is fine when the latency allows it. Timing enters where you **chose**, not through a register

Analogy to Multi-Threading

Shared variable



Message queues

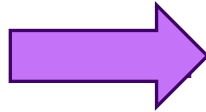


Hardware	Multi-threaded software
Free-running kernel	Thread (actor)
Kernel's internal state	Private variables
AXI4-Lite register written mid-run	Shared variable, no lock: a race
Host-activated kernel with registers	Set config, start thread, join
Command / response FIFO	Message queue / reply
Queue merge, shared memory port	Shared mailbox, shared resource

Go proverb: “Don't communicate by sharing memory; **share memory by communicating.**”

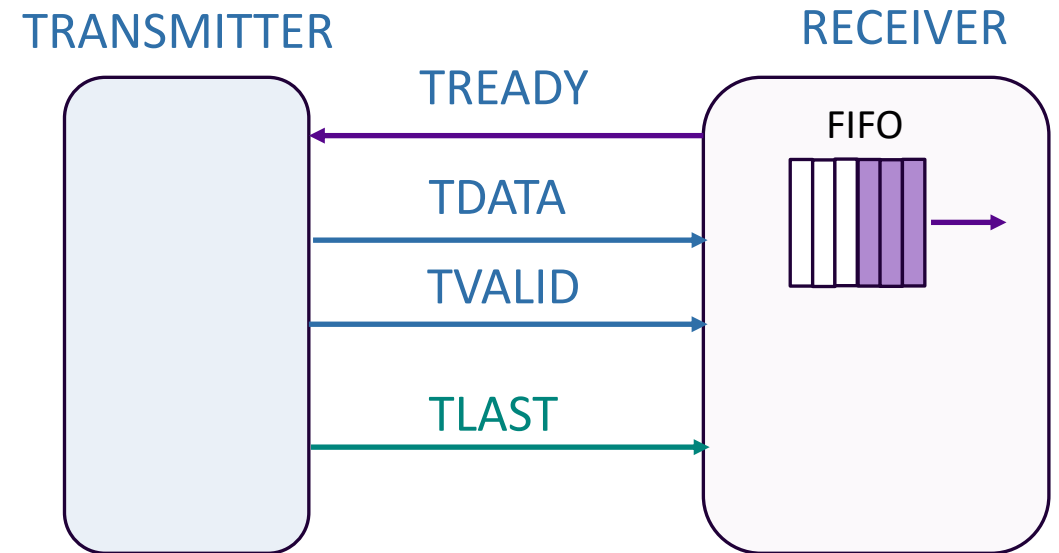
Outline

- FIFOs
- AXI4-Stream: Pure Streaming
- Command-Response Pattern with FIFOs

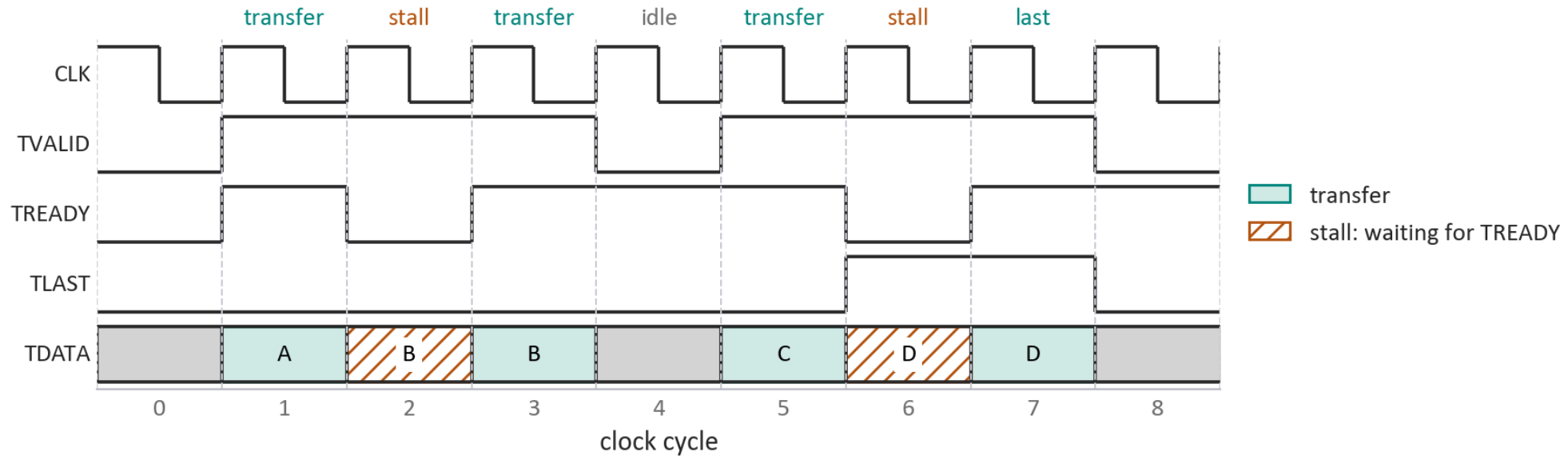
 Implementing Command-Response IPs in Vitis

AXI4-Stream Protocol with Packetization

- Add a **TLAST** line
- **TLAST** = 1 on the last transfer
- Indicates the end of packet
- Packets are transferred in **bursts**
 - Begins with first **TVALID** = 1
 - Ends with the transfer that has **TLAST**=1
- Cycles in a burst are called **beats**
 - Transfer beat (**TVALID**=**TREADY**=1)
 - Idle beat (**TVALID**=0)
 - Stall beat (**TVALID**=1, **TREADY**=0): data held
 - Last beat (**TVALID**=**TREADY**=**TLAST**=1)



Example Burst Timing



- A packet of four elements, A–D: **7 beats, 4 transfers**
- B and D stall: TREADY is low, and TDATA and TLAST are **held**
- Cycle 4 is idle: TVALID drops inside the packet

The packet ends with the **transfer** that has TLAST = 1: cycle 7, not cycle 6 where TLAST first rises.

AXI4-Stream Burst with TLAST

PRACTICE

Cycle	TREADY	TVALID	TLAST
0	0	0	0
1	1	1	0
2	0	1	0
3	1	1	0
4	0	1	1
5	1	1	1

A transfer occurs on any cycle where TVALID = 1 and TREADY = 1.

- (a) What is the first cycle of the burst, the first with TVALID = 1?
- (b) On which cycle does the last transfer occur?
- (c) How many elements were transferred in the burst?

Also in the LLM grader: Unit 5 › AXI4-Stream burst with TLAST

AXI4-Stream Burst with TLAST

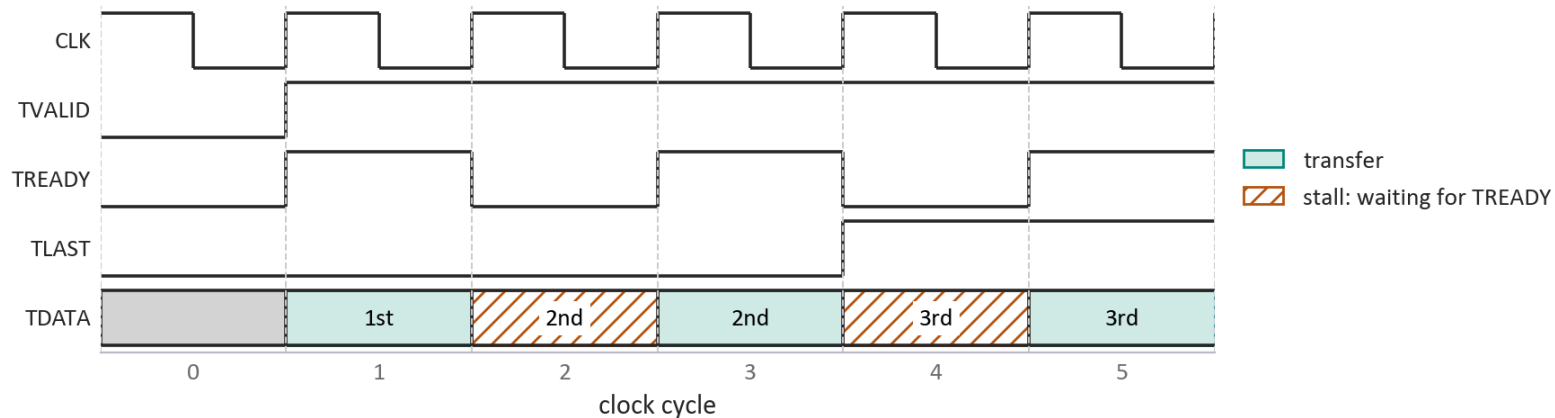
SOLUTION

- (a) **Cycle 1**: the first cycle with TVALID = 1
- (b) **Cycle 5**. TLAST rises in cycle 4, but TREADY = 0 there, so the last element stalls
- (c) **3 elements**: transfers on cycles 1, 3 and 5. The burst is 5 beats, 2 of them stalls

Common errors

Counting a cycle with TVALID = 1 but TREADY = 0: 5 elements.

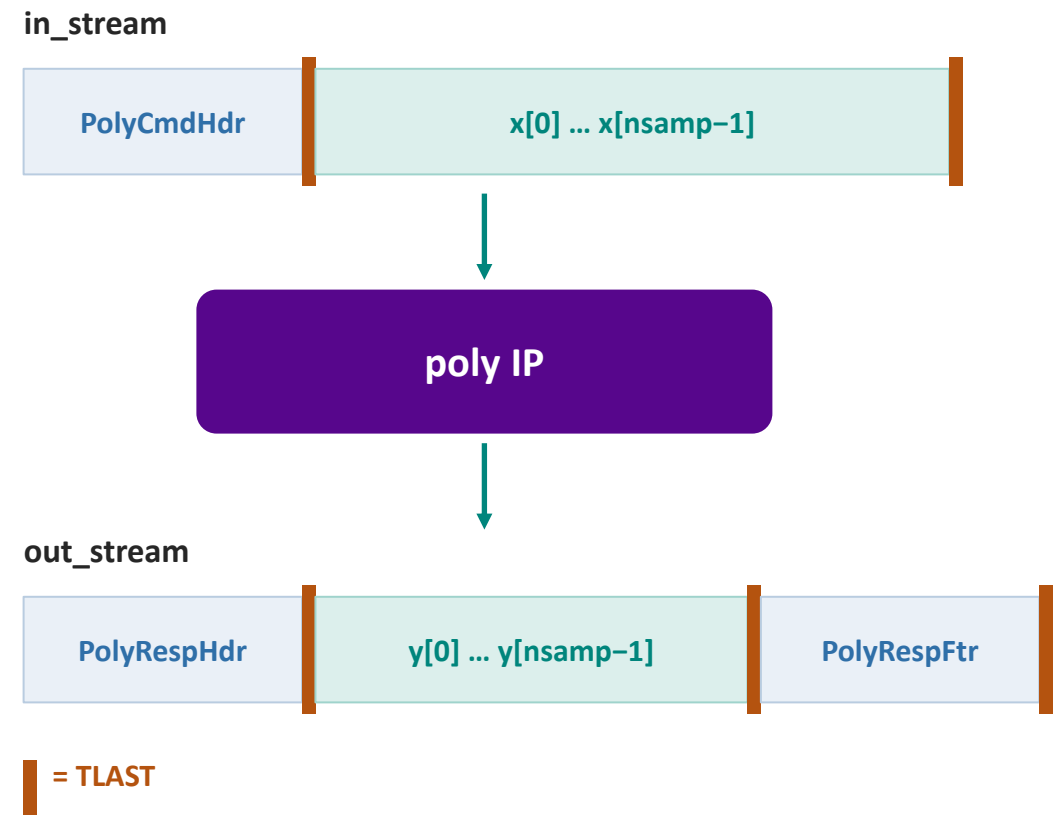
Ending the burst where TLAST first rises: cycle 4.



Also in the LLM grader: Unit 5 > AXI4-Stream burst with TLAST

Example: A Command-Response IP

- Evaluate a cubic on a stream of samples
 - $y[k] = c_0 + c_1x[k] + c_2x[k]^2 + c_3x[k]^3$
- The coefficients arrive **with the data**, in a command
 - Each transaction can use a different polynomial
 - No separate AXI4-Lite path to keep in step
- One **transaction**: a command in, a response out
- **TLAST** ends each message, so the IP can tell a header from the samples that follow



Command and Response Messages

Message	Field	Type	Holds
PolyCmdHdr	tx_id	uint16	Transaction ID
	coeffs	4 × float32	$c_0 \dots c_3$
	nsamp	uint16	Number of samples that follow
PolyRespHdr	tx_id	uint16	Echo of the command's ID
PolyRespFtr	nsamp_read	uint16	Samples actually read
	error	3-bit enum	What went wrong, if anything

Defined once, in Python

`poly_schema.py` describes each message as a waveflow data schema.

The C++ that packs and unpacks them is generated from it, so the kernel, the testbench and the model agree.

ONE TRANSACTION, WHERE EVERY | IS A TLAST

```
in_stream:  PolyCmdHdr  | x[0], x[1], ..., x[nsamp-1] |
out_stream: PolyRespHdr | y[0], y[1], ..., y[nsamp-1] | PolyRespFtr |
```

Robust Messages: IDs and Error Codes

Transaction IDs

- The command carries `tx_id`; the response header echoes it
- The host matches each response to its command
- A gap or a repeat in the IDs exposes a lost or duplicated message
- Several commands can be in flight at once

Error codes, in the footer

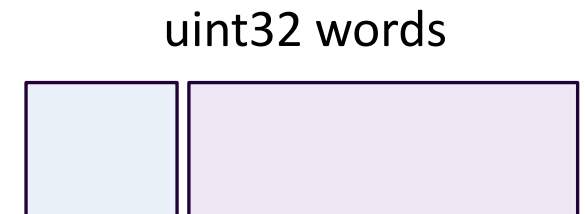
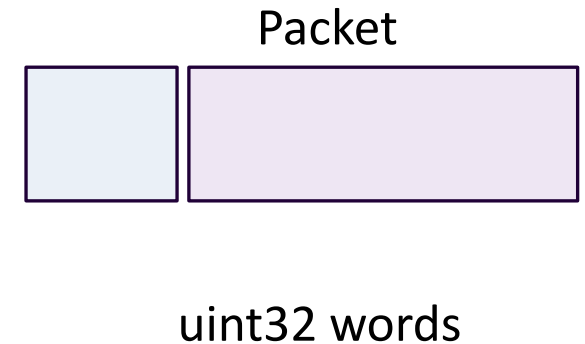
Code	Error
0	NO_ERROR
1	TLAST_EARLY_CMD_HDR
2	NO_TLAST_CMD_HDR
3	TLAST_EARLY_SAMP_IN
4	NO_TLAST_SAMP_IN
5	WRONG_NSAMP

Hardware cannot stop on bad input: the next command is already queued. It must **report** the error and **resynchronize** at the next TLAST.

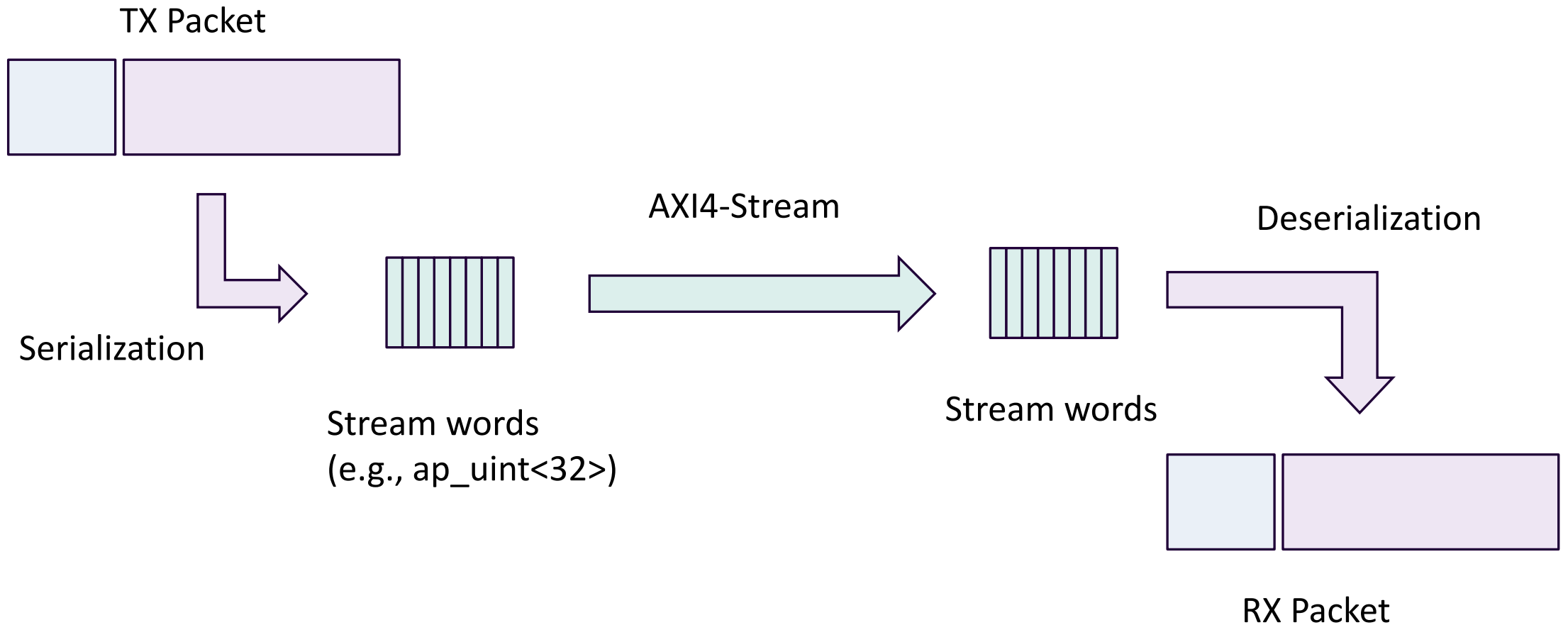
The demo's hand-written kernel reports; its general kernel also recovers.

Serialization / Deserialization

- Packets have arbitrary:
 - Structure (often different fields)
 - Arbitrary, sometimes variable, length
- Hardware streams have specific bit widths
 - Ex: 32, 64, 128, ...
 - Each stream element is a **word**
- **Serialization**: Converts packets to array of stream words
- **Deserialization**: Streams words to packets



Serialization / Deserialization Illustrated



Serialization / Deserialization in Hardware

- Python and C++ have widely-used serialization tools
- Ex: Python pickle
 - Can serialize an arbitrary class
 - Serializes complex nested structures
- Ex: Boost.serialization
 - Similar package in C++
- No standard library in hardware
- Written by hand, or generated from a schema

```
import pickle

# 1. The Python object to be saved (pickled)
data_to_save = {
    "name": "Jane Doe",
    "age": 30,
    "city": "Anytown",
    "is_active": True
}

# 2. Save the dictionary into a file
# Open a file in "write binary" ('wb') mode
with open('profile.pkl', 'wb') as file:
    # Use pickle.dump() to serialize the data into the file
    pickle.dump(data_to_save, file)
```

Packing Fields into Words

- A stream moves fixed-width **words**: 32 or 64 bits
- **Rule**: no field crosses a word boundary
 - Pack small fields together; start a new word when the next will not fit
 - Set unused bits to zero
- An integer field: assign its value to a bit range
 - `w.range(15, 0) = nsamp_read;`
- A **fixed-point** field: copy its **bits**, not its value
 - `w.range(11, 0) = a.range(11, 0);` copies the bits
 - `w.range(11, 0) = a;` converts the value: 0.75 becomes 0

The demo's PolyRespFtr: two fields in one 32-bit word



Its PolyCmdHdr at 32 bits: six words



A float fills a word exactly. A 16-bit field leaves the upper half of its word zero, which a 64-bit stream packs more tightly.

Serializing a Command Header

PRACTICE

`ap_uint<n>` is an n-bit unsigned integer. `ap_fixed<W,I>` is a signed fixed-point value with W bits in all, I of them integer bits, the sign included.

- (a) How many 32-bit words does one header need, if no field may be split across a word boundary?
- (b) Complete the function below to serialize a header into 32-bit words. Use the minimum number of words.

```
class CmdHdr {
    ap_uint<16> trans_id;
    // Q1.11: 1 sign bit, 11 fractional bits;
    // 1.0 corresponds to  $\pi$  radians
    ap_fixed<12,1> yaw, pitch, roll;
    ap_uint<16> nsamp;
};
```

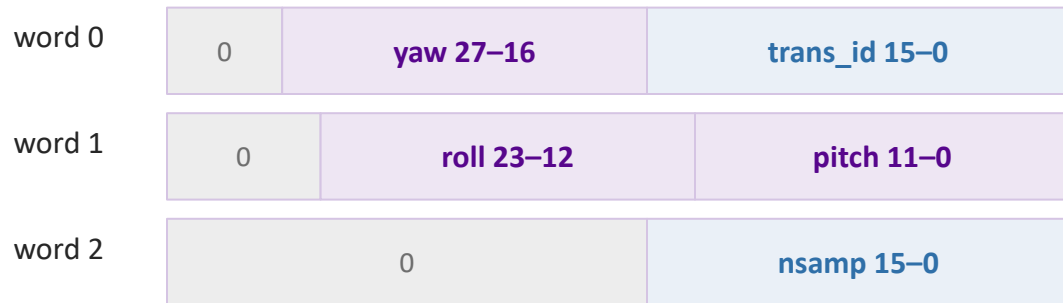
```
void serialize(const CmdHdr &cmd_hdr,
               ap_uint<32> words[/* ? */]) {
    ...
}
```

Also in the LLM grader: Unit 5 › Serializing a command header

Serializing a Command Header

SOLUTION

- (a) $16 + 12 + 12 + 12 + 16 = 68$ bits: more than two words.
Three suffice: **3 words**



The error that matters

`w0.range(27, 16) = cmd_hdr.yaw;` converts the value to an integer:
yaw = -0.25 packs as 0, not 0xE00.

```
void serialize(const CmdHdr &cmd_hdr,
               ap_uint<32> words[3]) {
    ap_uint<32> w0 = 0;           // trans_id | yaw
    w0.range(15, 0) = cmd_hdr.trans_id;
    w0.range(27, 16) = cmd_hdr.yaw.range(11, 0);
    words[0] = w0;

    ap_uint<32> w1 = 0;           // pitch | roll
    w1.range(11, 0) = cmd_hdr.pitch.range(11, 0);
    w1.range(23, 12) = cmd_hdr.roll.range(11, 0);
    words[1] = w1;

    ap_uint<32> w2 = 0;           // nsamp
    w2.range(15, 0) = cmd_hdr.nsamp;
    words[2] = w2;
}
```

Also in the LLM grader: Unit 5 › Serializing a command header

Deserializing a Command Header

PRACTICE

The same `CmdHdr` has been serialized into three 32-bit words with the packing in the table.

Complete `deserialize`. It must assign every field, and each fixed-point field must get back **exactly** the value serialized, negative angles included.

Word	Bits	Field	Width
0	15:0	trans_id	16
0	27:16	yaw	12
1	11:0	pitch	12
1	23:12	roll	12
2	15:0	nsamp	16

```
class CmdHdr {
    ap_uint<16> trans_id;
    // Q1.11: 1 sign bit, 11 fractional bits;
    // 1.0 corresponds to  $\pi$  radians
    ap_fixed<12,1> yaw, pitch, roll;
    ap_uint<16> nsamp;
};
```

```
void deserialize(CmdHdr &cmd_hdr,
                 ap_uint<32> words[3]) {
    // Fill in your solution here
}
```

Also in the LLM grader: Unit 5 › Deserializing a command header

Deserializing a Command Header

SOLUTION

Assigning an integer to an `ap_fixed` is a **numeric conversion**, not a bit copy.

`cmd_hdr.yaw = w0.range(27, 16);` reads the 12 bits as an integer between -2048 and 2047, which wraps to 0 or -1 in the range [-1, 1).

Write the bits into the field with `.range()` instead. All 12 bits, the sign bit included, are copied, so no sign extension is needed.

The same rule, both ways

Serialize: `w.range(...) = f.range(11, 0);`

Deserialize: `f.range(11, 0) = w.range(...);`

```
void deserialize(CmdHdr &cmd_hdr,
                 ap_uint<32> words[3]) {
    ap_uint<32> w0 = words[0];
    ap_uint<32> w1 = words[1];
    ap_uint<32> w2 = words[2];

    // Word 0: trans_id | yaw
    cmd_hdr.trans_id = w0.range(15, 0);
    cmd_hdr.yaw.range(11, 0) = w0.range(27, 16);

    // Word 1: pitch | roll
    cmd_hdr.pitch.range(11, 0) = w1.range(11, 0);
    cmd_hdr.roll.range(11, 0) = w1.range(23, 12);

    // Word 2: nsamp
    cmd_hdr.nsamp = w2.range(15, 0);
}
```

Also in the LLM grader: Unit 5 › Deserializing a command header

Hand-Written vs. Generated Serialization

BY HAND — poly32.cpp

```
ap_uint<32> resp_ftr = 0;
resp_ftr.range(15, 0) = nsamp_read;
resp_ftr.range(18, 16) = (unsigned) error;
write_word(out_stream, resp_ftr, true);
```

- The bit positions live in the kernel
- The testbench reads the same word with the generated class: bits 15:0 and 18:16 must match **by hand**
- Move a field, and every file that touches the message must move with it

FROM A SCHEMA — poly_schema.py

```
class PolyRespFtr(DataList):
    elements = {
        "nsamp_read": {"schema": Nsamp},
        "error": {"schema": PolyErrorField},
    }
```

kernel

poly.cpp
generated header

testbench

tb_poly.cpp
generated header

model

poly_golden.py
imports the schema

Defined **once**: change the schema, and every side that packs or unpacks the message follows.

The Demo: a Command-Response IP

sdrangan.github.io/hwdesign — Demos › Command-Response FIFO Interface

RUN THE WHOLE FLOW

```
cd demos/stream/poly
python poly_build.py --list-steps
python poly_build.py --through verify_csim \
    --kernel poly32 --force-step csim
python poly_build.py
```

WHAT TO WATCH FOR

- The messages are written once, in `poly_schema.py`; the headers that pack them are generated
- Three kernels, one function: `poly32` and `poly64` pack by hand, `general` uses the generated code
- Every TLAST is checked; a misplaced one is reported in the footer
- No start or done (`ap_ctrl_none`): coefficients arrive in a **command**

Three transactions, checked field by field against the Python model. Output starts 34 cycles after the input, then comes out one sample per cycle.

Same stream interface as the filter; the messages are what is new.

Choosing an Interface

	AXI4-Lite registers	AXI4-Stream, pure	AXI4-Stream, packets
Carries	A few control values	An endless sequence of samples	Commands and data, framed by TLAST
Rate	About 3 cycles per value	One value per cycle	One word per cycle
Control	Is the interface	A separate path, hard to keep in step	Travels with the data
Flow control	A handshake per transaction	TREADY backpressure into a FIFO	The same, plus IDs and error codes
Seen in	Unit 4, scalar function	Unit 5, streaming filter	Unit 5, polynomial IP

Registers for a **host-activated** job; streams for anything that **runs free**.